

RETROSPECTIVA CICLO 4

1. ¿Cuáles fueron los mini-ciclos definidos? Justifícalos.
 - Mini-ciclo 1: Jerarquía de ruedas y addWheel(type, pos). Wheel pasó a ser una clase abstracta con tres tipos (NormalWheel, LeftyWheel y RebelWheel), que cubren el requisito 17. El comportamiento que cambia en cada tipo quedó en métodos que se sobrescriben: girar (la lefty copia a su izquierda), setLocked (la rebel lo ignora), puedeIntercambiarse y puedeEliminarse. addWheel(String type, int pos) quedó con la firma del diseño del PDF.
 - Mini-ciclo 2: Jerarquía de símbolos. Symbol pasó a ser abstracta con NormalSymbol, EphemeralSymbol y ShySymbol, que cubren el requisito 18. SlotMachine crea cada tipo con crearSimbolo desde addSymbol(type, pos, color), y le avisa al símbolo cuando una rueda lo selecciona (avisarSeleccion llama a seleccionado). El ephemeral se encoge hasta quedar como un punto y el shy alterna entre visible e invisible.
 - Mini-ciclo 3: Tipo nuevo LazyWheel. Cubre el requisito 19: una rueda perezosa que solo se mueve una de cada dos veces en los giros aleatorios. Para que un símbolo no reaccione cuando su rueda se quedó quieta, Wheel.girar pasó a devolver un boolean (si se movió o no) y turnWheel solo avisa al símbolo en ese caso.
 - Mini-ciclo 5: Distinción visual de los tipos. Cubre el requisito de usabilidad. Cada símbolo declara su forma (círculo, cuadrado y triángulo) con un método abstracto getForma y Wheel dibuja la figura que corresponde con el color, el tamaño y la visibilidad del símbolo a través de dibujarRueda. Cada tipo de rueda tiene además su propio color de cuerpo (getColorCuerpo abstracto).
 - Mini-ciclo 6: Corrección de defectos de SlotMachine. Se revisó el código y se corrigieron cuatro cosas:
 - addSymbol movía los índices de las ruedas al insertar un símbolo antes de otros por lo que las ruedas cambiaban de símbolo sin que nadie las tocara.
 - Tipos y colores null lanzaban excepción.
 - pintarCelebracion ahora redibuja las ruedas con dibujarRueda.
 - spin(wheel, steps) actualiza el dibujo después de avisar a los símbolos.
 - Mini-ciclo 7: Pruebas de unidad y pruebas compartidas. SlotMachineC4Test cubre los tipos de rueda (rebel, lefty, lazy y colores distintos) los tipos de símbolo (formas, ephemeral con su mínimo, shy y estado por rueda) y los arreglos. SlotMachineCC4Test tiene tres pruebas compartidas.
 - Mini-ciclo 8: Constantes a enum. Las constantes numéricas (márgenes, separación, ancho de rueda, tamaño del símbolo, tamaño mínimo y paso del ephemeral) pasaron al enum Medida.
 - Mini-ciclo 9: Diagramas y cierre. Se actualizó el diseño en astah con las nuevas jerarquías de ruedas y símbolos y el enum. Se prepararon las pruebas de aceptación y se realizó la retrospectiva.

2. ¿Cuál es el estado actual del proyecto en términos de mini-ciclos? ¿Por qué?
Los 9 mini-ciclos planeados para el ciclo 4 están completos, implementados y funcionando.
3. ¿Cuál fue el tiempo total invertido por cada uno de ustedes? (Horas/Hombre)
Juan David Rojas Heredia invirtió 18 horas. Como el proyecto fue individual, el total es de 18 horas/hombre.
4. ¿Cuál consideran fue el mayor logro? ¿Por qué?
El mayor logro fue aplicar bien el polimorfismo. Antes, el comportamiento de cada tipo habría tenido que ir en if dentro de SlotMachine. Ahora cada tipo define el su comportamiento en su propia clase.
5. ¿Cuál consideran que fue el mayor problema técnico? ¿Qué hicieron para resolverlo?
El mayor problema fue el estado de los símbolos compartido entre ruedas. Hay un solo objeto de símbolo por color, así que si dos ruedas mostraban el mismo shy, el estado se compartía y se activaba dos veces, y el símbolo quedaba como al principio.
6. ¿Qué hicieron bien como equipo? ¿Qué se comprometen a hacer para mejorar los resultados?
Como trabajé de forma individual, lo que mejor hice fue revisar y corregir las entregas anteriores en vez de dejarlas como estaban. En este ciclo corregí defectos de SlotMachine de los ciclos previos. Lo que debo mejorar es la división de mi tiempo. Me comprometo a estimar el tiempo de cada mini-ciclo desde el inicio,
7. Considerando las prácticas XP incluidas en los laboratorios, ¿cuál fue la más útil? ¿Por qué?
La práctica XP más útil fue la refactorización (refactoring), que consiste en mejorar la estructura del código sin cambiar lo que hace. La apliqué varias veces en este ciclo. Pasé el comportamiento de cada tipo de rueda y de símbolo de condicionales a clases con polimorfismo. Me sirvió porque cada cambio de estos tocaba código de ciclos anteriores y habría sido muy riesgoso hacerlo sin saber si rompía algo.
8. ¿Qué referencias usaron? ¿Cuál fue la más útil? Incluyan citas con estándares adecuados.
La referencia más útil fue Claude, porque me explicó el código paso a paso, revisó mis clases y me ayudó a encontrar errores. Para las estructuras de datos y los conceptos del lenguaje usé la documentación oficial de Oracle. El uso de IA generativa quedó señalado en los comentarios del código. Las referencias, en formato APA 7, son estas:

- Anthropic. (2026). *Claude* (Claude Sonnet 5.5). <https://claude.ai>
- OpenAI. (2023). *ChatGPT*. <https://chat.openai.com>